

Final exam: marking guide

For the instructor. Do not hand this out.

Section A: answers

30 marks

One mark per question. Where a question has two or three parts, split the mark between them and round the section total to the nearest whole mark. Accept any wording that shows the student understands the point. The notes say what to accept and what not to.

HTML

1. ``
Half for the path, half for alt text that describes the image. `alt="image"` or `alt="logo.png"` does not get the alt half.
2. "See all products" is a link (`<a href="products.html">`) because it takes you somewhere. "Add to cart" is a `<button>` because it does something on the page. The reason is needed for the full mark.
3. The headings skip from h1 to h3, and h3 comes before the h2 it should sit under. Screen reader users move around a page by its headings, so the outline they hear is wrong or confusing. Fix: Rice dishes and Soups should both be h2. Half for spotting the problem, half for why it matters.
4. `#prices` looks for an element with `id="prices"`, and the section only has a class. Fix: `<section id="prices">`. Changing the link to `.prices` is wrong.
5. (b), the viewport meta tag.
6. Any two of: screen readers and other assistive tools use them to describe and jump around the page; search engines understand the page better; other developers, and you later, can read the structure without decoding a wall of divs; they give clear hooks for CSS. "It looks better" is not a reason, since they look the same.
7. (a) `<blockquote>` (b) `<figure>` with `<figcaption>` (c) `<ol>` with `<li>`. One third each.
8. A `<p>` cannot sit directly inside a `<ul>`; only `<li>` can, so Fried rice should be an `<li>`. And `<strong>` is closed after `</li>`, so the tags overlap; it should be `<strong>(new)</strong></li>`. Half each.

CSS

9. (b). (a) starts at the root of the site, not the folder the page is in, and `<style>` and `<script>` cannot load a stylesheet.
10. (a) Red. An id beats a class. (b) Blue. Both `.note` rules are equally specific, so the one that comes later wins. Half each; (b) needs the reason.

11. (a) 350px: 300 + 20 + 20 of padding + 5 + 5 of border. Margin is outside the border, so it does not count. (b) 300px. Half each.
12. display: flex goes on the parent, .cards, not on the children. Flex arranges an element's children.
13. A row. A media query adds no extra weight. Both rules use .cards, so the one lower in the file wins, and it sets row at every width. Fix: put the media query after the main rule. A student who says "column" gets nothing.
14. (b) transform-origin: bottom center;
15. overflow: hidden on the section, the circle's container. Putting it on the circle itself, or on the body, does not get the mark.
16. 2rem is 32px and 4rem is 64px. (a) 6vw is 24px, below the minimum, so 32px. (b) 6vw is 60px, so 60px. (c) 6vw is 84px, above the maximum, so 64px. One third each.
17. The top 60% of the photo shows normally. From there it fades out until it is fully see-through at the bottom edge, so whatever is behind it shows through. "It fades towards the bottom" is enough for the mark if the student is clear that the top part is untouched.
18. That declaration is not being applied because another rule overrides it. Likely causes: a more specific selector, or a rule with the same specificity that comes later. Also accept: the value is invalid or misspelled (DevTools usually adds a warning icon for that).

JAVASCRIPT

19. 53 2 number. "5" + 3 joins text; minus and times turn the strings into numbers. All three are needed.
20. It throws TypeError: Assignment to constant variable. A const cannot be given a new value. Fix: declare it with let.
21.

```
console.log(`${name} ordered ${items} items for ${items * price} naira.`);
```

Must use backticks and work out the 4500 inside \${}. Typing 4500 in directly gets half.
22. 8, then undefined. The function prints the answer but does not return it, so result gets undefined. Half for the two lines, half for the explanation.
23. (a) true (b) false (c) false. Use ===, because == changes the types before comparing and gives surprises like (a). === checks the value and the type.
24. (a) "Welcome" (b) "Almost" (c) "Sorry". One third each. A common slip is (a) "Almost" from reading >= as >.
25. 3700 (1200 + 2500; 800 and 500 are under 1000).
26. i <= names.length runs one time too many. On the last pass i is 3, names[3] is undefined, and reading .length of undefined throws. Fix: i < names.length.

27.

```
for (const student of students) {  
  if (student.score >= 50) {  
    console.log(student.name);  
  }  
}
```

Also accept a counting loop with `students[i].score`, or `forEach`. It must use `>=`, and log the name, not the whole object. Half if it logs the whole object.

28. The selector asks for an id, `#buy-btn`, but the button only has a class. `querySelector` finds nothing and returns `null`. Fix: use `".buy-btn"`, or give the button `id="buy-btn"`.

29. The dot belongs in the CSS selector, not the class name. This adds a class literally called `".error"`, which `.error` in the CSS does not match. Fix: `classList.add("error")`.

30. `input.value` is read once, when the page loads and the box is still empty, so `typed` is always `""`. Fix: read the value inside the click function, for example `const typed = input.value;` as its first line. Half for the cause, half for the fix.

Section B: marking the practical

70 marks

Mark each line in the tables on the exam paper. Give part marks freely; a feature that mostly works should get most of its marks. These quick tests cover nearly everything. Open the page with the console showing.

Testing the booking panel

1. Press plus on Regular twice and on Student three times. With the starting prices the page should show 10500 before the discount, 9450 after it, and "Group discount applied".
2. Press minus on Student until it stops. It must stop at 0, and the discount must disappear once the order drops under 5 tickets.
3. Add tickets until the order has 10, then press plus again. The count stays at 10 and the limit message appears. Check this across different ticket types, not just one: 4 Regular, 4 VIP and 2 Student, then one more of any type.
4. Press Clear, then Book. Expect "Pick at least one ticket."
5. Add one ticket, type a one-letter name and Book. Expect "Enter your name." Then type a proper name with a 10-digit phone number. Expect "Enter an 11-digit phone number." Then fix the number. Expect the success message with the right count and total, and check that the red error style is gone.
6. Change one price in the array in `script.js` and reload. If the total does not change, the total is not coming from the array, so the data line gets 0 and the function line loses at least half.

Testing the rest

- Click every nav link.
- Use the device toolbar at 360px and 600px, or check on a phone. Scroll sideways at 360px; if the page moves, the responsive line loses at least 2.
- Press Tab through the page. Links and buttons need a visible focus style.
- Search the HTML for `style=""`. Any match costs the inline-styles mark.
- Read the total function. It must loop over the array and return a number. A function that only sets text on the page and returns nothing gets at most 3 of the 6.
- Check that the Book handler reads `.value` inside the click function. Reading it once at the top of the file is the bug from question 30 and costs 2 marks.

Common deductions

What you see	Suggested deduction
Prices hard-coded into the total, for example <code>count1 * 3000</code>	Data line 0; total function at most 3 of 6
Limit of 10 checked per ticket type instead of for the whole order	Minus 3 from plus and minus
Discount worked out but only the new price shown	Minus 1
All three Book problems shown at once instead of the first one	Minus 2
<code>.style.color</code> used instead of the error and success classes	Minus 2
Divs or spans used as buttons	Minus 1 from the booking panel line
Red errors in the console during normal use	Quality line 0

notes.txt

Two marks per answer. Give 2 when the answer is specific to the student's own code, 1 when it is correct but general enough to have been written without seeing the project, and 0 when it is missing or wrong. For question 2 the student should mention the loop, adding each price times its count, and returning the result. For question 3 they should describe each property in the technique they used, as it behaves on their page.

When you ask a student to explain

Pick two or three lines from their `script.js`, usually the total function and the Book handler, and ask what each one does and what would happen if it were removed. A student who wrote the code can answer quickly even if they are nervous. If they cannot explain a part at all, take away the marks for that part.